PYTHON PROGRAMMING

Basics · Types · Socket Programming · Real-World Applications

Table of Contents

Chapter 1 — Introduction to Computer Networking

- What is Networking?
- Why Networking is Needed
- How Computers Communicate
- Real-World Examples

Chapter 2 — Networking Basics

- Client & Server
- Request & Response
- IP Address & Port Number
- Protocol & Data Transfer

Chapter 3 — Types of Networking

- LAN · MAN · WAN · Internet

Chapter 4 — Python Networking & Socket Module

- What is Socket Programming?
- The socket Module
- Important Socket Functions

Chapter 5 — TCP Client-Server Program

- Server Code
- Client Code
- Line-by-Line Explanation
- Workflow Diagram

Chapter 6 — Code Deep-Dive

- recv() · Buffer Size · decode()
- Full Walkthrough

Chapter 7 — Real-Time Applications

- Chat Apps · Games · File Transfer · IoT

Chapter 8 — TCP vs UDP Comparison

Chapter 9 — Mini Project Ideas

Chapter 10 — Summary & Key Points

Chapter 1

Introduction to Computer Networking

What is Networking?

A **computer network** is a group of two or more computers or devices connected together so they can share information and resources. Think of it like a postal system — instead of letters, computers send packets of data to each other through wires, Wi-Fi, or the internet.

■ Real-life analogy: When you send a WhatsApp message, your phone breaks the message into tiny packets, sends them over the internet, and your friend's phone reassembles those packets back into the message.

Why Networking is Needed

■ **Share Resources**	Multiple computers can share a printer, files, or internet connection.
■ **Fast Communication**	Messages, emails and files travel across the world in milliseconds.
■ **Access Information**	The internet is a global network that gives access to information from anywhere.
■ **Online Collaboration**	Games, video calls, and team projects all rely on networking.
■ **Data Backup**	Files can be stored and retrieved from remote servers (cloud).

How Computers Communicate With Each Other

Computers communicate by following strict rules called **protocols**. The most common is **TCP/IP** (Transmission Control Protocol / Internet Protocol). Here is the basic flow:

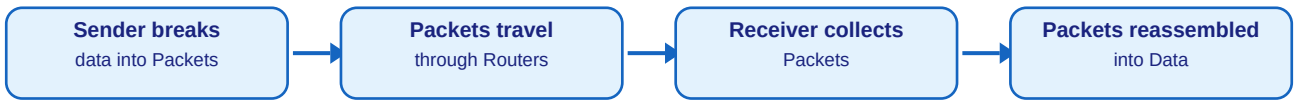

Fig 1.1 – How data travels across a network

Real-World Networking Examples

■ WhatsApp Message

You type a message → app breaks it into packets → packets travel via internet → friend's phone receives packets → message appears on screen.

■ Browsing a Website

You type www.google.com → your browser sends a request to Google's server → server sends back the webpage → browser displays it.

■ Online Games

Your game sends your actions (move, shoot) to the game server → server updates all players → your screen refreshes in real-time.

■ File Sharing

You upload a file to Google Drive → it is sent over the network to Google's servers → your friend downloads it from there.

Chapter 2

Networking Basics

Client and Server

Every network communication involves two roles: a **client** and a **server**.

CLIENT	SERVER
Sends a request for data or service. Example: Your web browser.	Receives the request and sends back a response. Example: Google's web server.
■■ Your laptop / phone	■ A powerful computer in a data centre

Request and Response

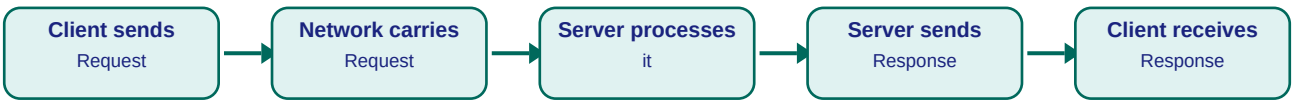

Fig 2.1 – Request / Response cycle

IP Address

An **IP Address** (Internet Protocol Address) is a unique number assigned to every device on a network — just like a home postal address. Without an IP address, the network does not know where to deliver data.

IPv4	4 numbers (0–255)	192.168.1.10
IPv6	8 groups of hex	2001:db8::1
Localhost	Special address	127.0.0.1 (your own PC)

Port Number

A **port number** is like an apartment number inside a building (IP address). One computer runs many services — each service listens on a different port.

Port	Service
80	HTTP – Web pages
443	HTTPS – Secure web
25	SMTP – Email

22	SSH – Remote login
65432	Custom Python programs

Protocol

A **protocol** is a set of rules that computers agree on before communicating. Without protocols, two computers cannot understand each other — like two people speaking different languages.

■ Common Protocols: • TCP – Reliable, ordered delivery (web, email, file transfer) • UDP – Fast but no guarantee (video streaming, gaming) • HTTP/HTTPS – Web browsing • FTP – File transfer

Data Transfer

Data is transferred in small units called **packets**. Each packet contains a header (source/destination address) and payload (actual data). They travel independently and are reassembled at the destination.

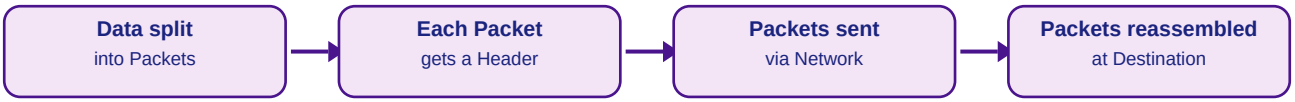

Fig 2.2 – Data Packet Flow

Chapter 3

Types of Networking

Networks are categorised by their **geographic size** — from a single room to the entire planet.

■ LAN – Local Area Network

Covers a small area like a home, school, or office.

Examples: School computer lab | Home Wi-Fi router | Office network

Speed: Very Fast (up to 10 Gbps)

■■ MAN – Metropolitan Area Network

Covers a city or large campus. Connects multiple LANs.

Examples: City-wide cable TV | University campus network | City Wi-Fi

Speed: Fast (up to 1 Gbps)

■ WAN – Wide Area Network

Covers countries or continents. Uses telephone lines, fibre, satellites.

Examples: Company branch offices connected across India | Bank ATM networks

Speed: Slower than LAN

■ Internet – Global Network of Networks

The biggest network in the world — a network of millions of networks.

Examples: Browsing YouTube | Sending Gmail | Zoom calls

Speed: Varies by connection

Network	Coverage	Example
LAN	Room / Building	Your school lab
MAN	City / Campus	University network
WAN	Country / Continent	Company branch offices
Internet	Entire World	Google, YouTube, WhatsApp

Fig 3.1 – Network types by geographic coverage

Chapter 4

Python Networking & Socket Module

What is Socket Programming?

A **socket** is one end of a two-way communication link between two programs running on a network. Think of a socket like a telephone — you need two phones and a phone number (IP + Port) to make a call.

■ Socket = IP Address + Port Number Example: 127.0.0.1 : 65432 Just like: Building Address : Flat Number

Python's socket Module

Python provides a built-in **socket** module. You do not need to install anything — it comes with Python!

```
#How to use it

import socket # Import the built-in socket module
```

Important Socket Functions

socket()

Creates a new socket object. Think of this as 'buying a new phone'.

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# AF_INET = IPv4, SOCK_STREAM = TCP
```

bind()

Assigns an IP address and port to the socket. Like 'registering your phone number'.

```
s.bind(("127.0.0.1", 65432))

# Server must call bind() before listening
```

listen()

Puts the server socket in listening mode — ready to accept connections.

```
s.listen(5)

# 5 = maximum number of waiting connections (backlog)
```

accept()

Waits for a client to connect. Returns a new socket for that client.

```
conn, addr = s.accept()
```

```
# conn = new socket for this client | addr = client IP, port
```

connect()

Used by client to connect to a server.

```
s.connect(("127.0.0.1", 65432))

# Client connects to server at IP 127.0.0.1 port 65432
```

send()

Sends data through the socket. Data must be bytes, not a string.

```
s.send(b"Hello Server")

# b notation converts string literal to bytes
```

recv()

Receives data from the socket. You specify the buffer size in bytes.

```
data = s.recv(1024)

# 1024 = receive up to 1024 bytes at a time
```

close()

Closes the socket connection. Always close sockets when done.

```
s.close()

# Releases the port and frees resources
```

Chapter 5

TCP Client-Server Program in Python

Workflow Overview

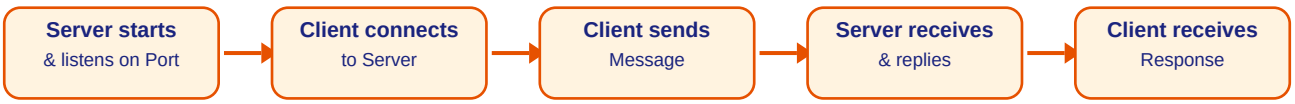

Fig 5.1 – TCP Client-Server Workflow

Server Code

The server is the program that **waits** for a client to connect. Run this program first in one terminal window.

```
# ■■■ server.py ■■■ Run this FIRST ■■■

import socket


# Step 1: Create a TCP socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)


# Step 2: Bind to an IP address and port

server.bind(('127.0.0.1', 65432))


# Step 3: Start listening for connections

server.listen()

print('[Server] Waiting for a connection...')


# Step 4: Accept a client connection

conn, addr = server.accept()

print(f'[Server] Connected by {{addr}}')


# Step 5: Receive a message from the client

data = conn.recv(1024)

message = data.decode()

print(f'[Server] Received: {{message}}')
```

```
# Step 6: Send a reply back to the client
conn.send('Hello from Server!' .encode())

# Step 7: Close the connection
conn.close()
server.close()
print('[Server] Connection closed.')
```

```
■ Output:
[Server] Waiting for a connection...
[Server] Connected by ('127.0.0.1', 54321)
[Server] Received: Hello from Client!
[Server] Connection closed.
```

Client Code

The client is the program that **connects** to the server and sends a message. Run this in a second terminal window after the server is running.

```
# ■■■ client.py ■■■ Run this SECOND ■■■
import socket

# Step 1: Create a TCP socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Step 2: Connect to the server
client.connect(('127.0.0.1', 65432))
print('[Client] Connected to server!')

# Step 3: Send a message to the server
client.send('Hello from Client!' .encode())

# Step 4: Receive the server's reply
reply = client.recv(1024).decode()
print(f'[Client] Server replied: {{reply}}')
```

```
# Step 5: Close the connection
client.close()
print('[Client] Connection closed.')
```

```
■ Output:
[Client] Connected to server!
[Client] Server replied: Hello from Server!
[Client] Connection closed.
```

Step-by-Step Explanation

Step	What Happens	Code
1	Server creates socket	`socket.socket(...)`
2	Server binds to port 65432	`server.bind(...)`
3	Server listens for connections	`server.listen()`
4	Client creates socket	`socket.socket(...)`
5	Client connects to server	`client.connect(...)`
6	Server accepts connection	`conn, addr = server.accept()`
7	Client sends message	`client.send(...encode())`
8	Server receives and prints message	`data = conn.recv(1024)`
9	Server sends reply	`conn.send(...encode())`
10	Client receives reply	`reply = client.recv(1024).decode()`
11	Both sides close connection	`.close()`

Chapter 6

Code Deep-Dive – Understanding Every Detail

Understanding recv(1024)

```
message = client.recv(1024).decode()
```

recv()

recv() stands for 'receive'. It waits until data arrives and then returns the raw bytes that were sent. The program pauses at this line until data comes.

1024 (Buffer Size)

1024 means 'receive up to 1024 bytes at a time'. One byte = one character roughly. So 1024 bytes is about 1 KB of data. For small messages this is fine. For large files you would use a loop.

.decode()

Converts bytes to a readable Python string. Computers send data as bytes (numbers 0-255). .decode() turns those bytes back into text using the UTF-8 encoding.

The encode() / decode() Pair

Sockets can only transfer **bytes**, not strings. So you always need to convert:

```
# String → Bytes (before sending)

message = 'Hello'

bytes_data = message.encode() # b'Hello'


# Bytes → String (after receiving)

received_bytes = b'Hello'

text = received_bytes.decode() # 'Hello'
```

AF_INET and SOCK_STREAM Explained

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Parameter	Meaning	When to use
AF_INET	Use IPv4 addresses	Most common - use this always

`AF_INET6`	Use IPv6 addresses	Newer internet standard
`SOCK_STREAM`	TCP – reliable, ordered	Web, email, file transfer
`SOCK_DGRAM`	UDP – fast, no guarantee	Video streaming, gaming

Complete Communication Sequence

Fig 6.1 – Full TCP handshake and message exchange

Chapter 7

Real-Time Applications Using Python Networking

Socket programming is the foundation of almost every networked application you use daily. Here are some important examples:

■ Chat Applications

Apps like WhatsApp and Telegram use sockets under the hood. Each user's device maintains a persistent socket connection to the server. When you type a message, it is sent via send(). The server forwards it to your friend's socket using recv().

- Multi-threaded server handles many clients simultaneously
- Messages encoded as JSON and sent as bytes
- Python libraries: socket, threading

■ Multiplayer Games

Online games like PUBG or Minecraft use networking for real-time player synchronisation. Game actions (move, shoot) are sent as UDP packets for speed. The game server processes all player data and broadcasts updates.

- UDP used for speed (small delay is acceptable)
- Server maintains the 'game state' for all players
- Python game networking: pygame + socket

■ File Transfer Systems

FTP (File Transfer Protocol) is built on sockets. Python can build a simple file transfer program using socket.send() in a loop until the entire file is transmitted.

- File split into chunks (e.g. 4096 bytes each)
- Each chunk sent one at a time
- Receiver writes chunks to a new file

■■ Remote Communication

SSH (Secure Shell) allows you to control another computer remotely. Python's paramiko library wraps SSH. Commands you type are sent over an encrypted socket to the remote computer.

- Used in DevOps and server management
- Python library: paramiko
- All traffic is encrypted

■■ IoT Devices

Smart devices (temperature sensors, smart bulbs) send data to a central server using lightweight socket connections. Python on a Raspberry Pi can connect to a server and send sensor readings every few seconds.

- Devices use TCP or MQTT protocol
- Python runs on microcontrollers (MicroPython)
- Applications: smart home, agriculture, healthcare

Chapter 8

TCP vs UDP – Comparison

Both TCP and UDP are transport-layer protocols but they work differently. Choosing the right one depends on whether you need **reliability** or **speed**.

Feature	TCP	UDP
Full Name	Transmission Control Protocol	User Datagram Protocol
Connection	Connection-oriented (handshake required)	Connectionless (no handshake)
Reliability	■ Guaranteed delivery Packets re-sent if lost	■ No guarantee Packets may be lost
Speed	■ Slower (acknowledgements needed)	■ Faster (no waiting for ack)
Order	■ Packets arrive in order	■ Order not guaranteed
Error Checking	■ Yes	■ Basic checksum only
Flow Control	■ Yes	■ No
Python Socket	SOCK_STREAM	SOCK_DGRAM
Use Cases	Web browsing (HTTP) Email (SMTP) File transfer (FTP) Banking apps	Video streaming Online gaming DNS queries Live video calls
Analogy	■ Registered Post (You get a receipt)	■ Radio broadcast (No confirmation)

Table 8.1 – TCP vs UDP Comparison

■ Rule of thumb: Use TCP when accuracy matters more than speed (banking, email, files). Use UDP when speed matters more than accuracy (games, video, live streaming).

Chapter 9

Mini Project Ideas for Beginners

Once you understand basic socket programming, these beginner projects will help you practice and build confidence.

Project 1: ■ Simple Chat Application

Build a two-way chat program where two computers can send messages to each other in real time.

- Server broadcasts messages to all connected clients
- Clients send messages via input()
- Use threading so server can handle multiple clients

Libraries: socket, threading | Difficulty: Beginner – 50 lines

Project 2: ■ File Sharing System

Build a program that sends a file from one computer to another over a local network.

- Client opens a file and reads it in chunks (e.g. 4096 bytes)
- Each chunk is sent using socket.send()
- Server receives chunks and writes them to a new file

Libraries: socket, os | Difficulty: Beginner-Intermediate – 70 lines

Project 3: ■ Client-Server Calculator

Client sends two numbers and an operation (+, -, *, /) to the server. Server calculates the result and sends it back.

- Client: input numbers and operator
- Server: eval() or if/else to calculate
- Great practice for request-response pattern

Libraries: socket, json | Difficulty: Beginner – 40 lines

Project 4: ■ Network Monitoring Tool

Scan your local network to find which IP addresses are active and which ports are open.

- Use socket.connect_ex() to test each port
- Loop through IP range to ping devices
- Print a report of active hosts

Libraries: socket, threading, ipaddress | Difficulty: Intermediate – 80 lines

■ Tip for students: Start with Project 3 (Calculator) as it is the simplest. Once that works, move to Project 1 (Chat App).

Chapter 10

Summary & Key Points to Remember

Congratulations! You have completed the Networking in Python guide. Here is a quick recap of everything important:

- **What is a Network?**
  A network connects computers so they can share data and resources.
- **Network Types**
  LAN (small area) → MAN (city) → WAN (country) → Internet (world).
- **Client & Server**
  Client sends requests. Server receives requests and sends responses.
- **IP & Port**
  IP = device address. Port = which program on that device.
- **Protocol**
  Rules computers follow to communicate. Most common: TCP/IP.
- **Sockets**
  A socket = IP + Port. Python uses socket module (built-in). socket.socket(AF_INET, SOCK_STREAM) creates a TCP socket.
- **Key Functions**
  bind() → listen() → accept() on server side. connect() → send() → recv() on client side.
- **encode() / decode()**
  Sockets send bytes, not strings. Use .encode() before sending and .decode() after receiving.
- **recv(1024)**
  Receive up to 1024 bytes. For large data, use a loop.
- **TCP vs UDP**
  TCP: reliable but slower. UDP: fast but may lose packets. Use TCP for accuracy, UDP for speed.
- **Real Applications**
  Sockets power: chat apps, online games, file sharing, IoT, remote control.

Keep Practising!

The best way to learn networking is to write code. Start with the simple client-server program, then build the chat application. Every great network engineer started exactly where you are now. You can do it! ■